

FAST

FORMAÇÃO ACCELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R

Testes de API

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



Boas vindas e introdução aos Testes de API com Postman

Testes de API
Aula 1



Olá! Eu sou Vitor Felix



Engenheiro de Qualidade de Software atuando no Cesar há pouco menos de 1 ano, e com 5 anos de experiência na área. Graduado em Engenharia da Computação pela POLI/UPE, certificado pela ISTQB com a CTFL (Certified Tester Foundation Level), e pós-graduando em Testes Ágeis pela CESAR School.

LinkedIn 

<https://www.linkedin.com/in/vitor-felix-603571161/>



Olá! Eu sou Luan Fonseca

Engenheiro de Qualidade de Software, atuando no Cesar há mais de 3 anos, e com 6 anos de experiência na área. Graduado em Sistemas de Informação pela UFRN, certificado pela ISTQB com a CTFL (Certified Tester Foundation Level), pós-graduado em Teste de Software, pela projeto Motorola/CIn-UFPE, pós graduando em Liderança Técnica pela CESAR School.



<https://www.linkedin.com/in/luanof/>

Estrutura do curso

Conteúdo:

Aula 1:

- O que é uma API e por que testá-la?
- Tipos de testes em APIs.
- Introdução ao Postman.
- Criando requisições e validando respostas no Postman.
- Testes manuais em API
- Scripts básicos de automação no Postman

Aula 2:

- Revisão da aula anterior
- O que é RestAssured e como configurar o ambiente
- Composição de um teste automatizado com RestAssured
- Requisições HTTP e como validá-las
- Validações robustas em API
- Autenticação em testes automatizados

Aula 3:

- Revisão das aulas anteriores
- Padrões e boas práticas na automação
- CI e sua integração com a qualidade
- Como executar os testes automatizados fora da IDE
- Testes de API e ferramentas de CI/CD
- Próximos passos no aprendizado

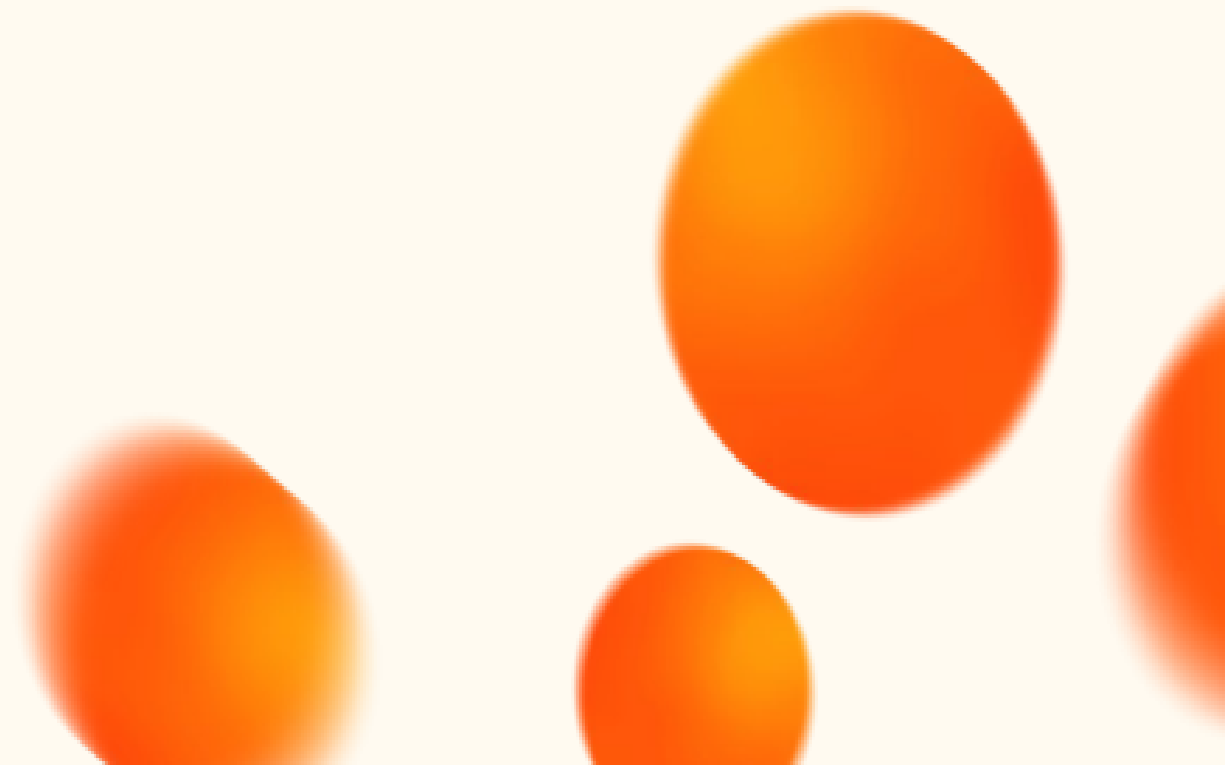
Intervalo: 15 à 20 minutos

Material das aulas: NEO

Avaliação Final: 5 questões objetivas

Vamos nos conhecer um pouco melhor?

<https://www.menti.com/ali4awqmp1s7>



O que veremos hoje?

- O que é uma API e por que testá-la?
- Tipos de testes em APIs
- Introdução ao Postman
- Criando requisições e validando respostas no Postman
- Testes manuais em API
- Scripts básicos de automação no Postman

O que é uma API e por que devemos testá-la?

API (Application Programming Interface)

- É um **conjunto de regras e protocolos** que permite que diferentes softwares se comuniquem sem precisar saber como eles foram implementados.
- Simplifica a integração e o desenvolvimento, pois **padroniza a forma de interação**.
- É a "**ponte**" que conecta diversas aplicações que usamos todos os dias (apps de clima, apps de transporte, bancos, e-commerce).



API: Um Contrato de Comunicação

Imagine a API como um acordo formal entre sistemas:

- **Quem Pede (o Consumidor da API):** É como você pedindo um serviço. Você precisa saber o que pode pedir e como.
- **Quem Oferece (a Própria API):** É como a empresa que oferece o serviço. Ela se compromete a entregar algo específico.

Este "contrato" define claramente:

- O que podemos pedir
- Como devemos pedir
- O que vamos receber

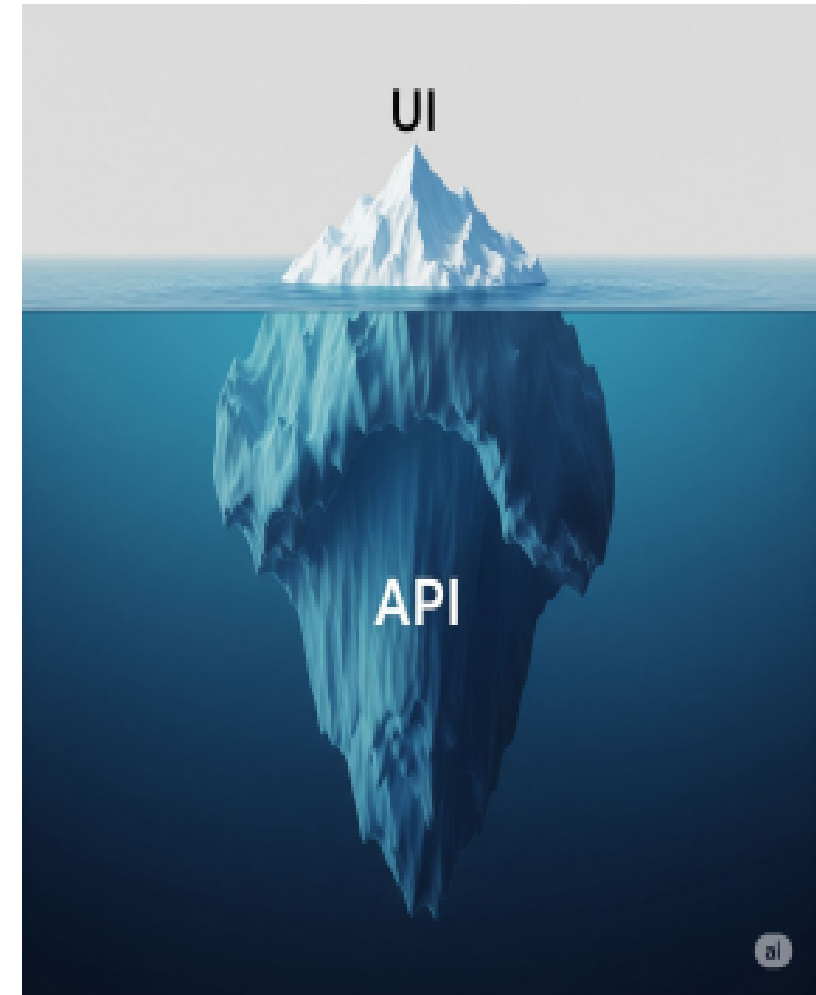
API: Um Contrato de Comunicação

Por Que Esse Contrato é Crucial?

- **Previsibilidade:** Garante que o sistema que usa a API saiba exatamente o que esperar do outro.
- **Estabilidade:** Se o "contrato" for alterado, ambas as partes precisam estar cientes para não "quebrar" a comunicação.
- **Qualidade:** Nos testes de API, nós verificamos se a API está cumprindo seu contrato. Isso significa que ela entrega o que promete, da forma que promete.

Por que testar API?

- **A API é o "Coração" do Software:** A maioria das funcionalidades e a lógica de negócio residem nela.
- **Deteção Precoce de Bugs:** Encontre problemas na base antes que cheguem ao usuário, tornando a **correção mais rápida e barata**.
- **Qualidade Robusta:** Garante a **estabilidade, performance e segurança** da aplicação, independentemente da interface.
- **Agilidade:** Testes de API são mais **rápidos e confiáveis** para automação e integração em pipelines de desenvolvimento.



Tipos de testes em APIs

Testes Funcionais

- **O que validam:** Se a API faz o que deveria fazer. As operações (criar, ler, atualizar, deletar dados, também conhecidos como **CRUD**) funcionam corretamente? Os dados de entrada são processados como esperado?
- **Exemplos:** Enviar um novo usuário e verificar se ele foi criado; buscar um produto e confirmar se os dados estão corretos; tentar logar com senha errada e verificar a mensagem de erro.

Testes de Contrato

- **O que validam:** Se a API está seguindo o seu "contrato" (a especificação) com outras aplicações. A estrutura das requisições e respostas está de acordo com o que foi definido?
- **Exemplos:** Verificar se um campo obrigatório está presente na resposta; se o tipo de dado de um campo é o esperado (número, texto); se os status codes estão corretos para cada cenário.

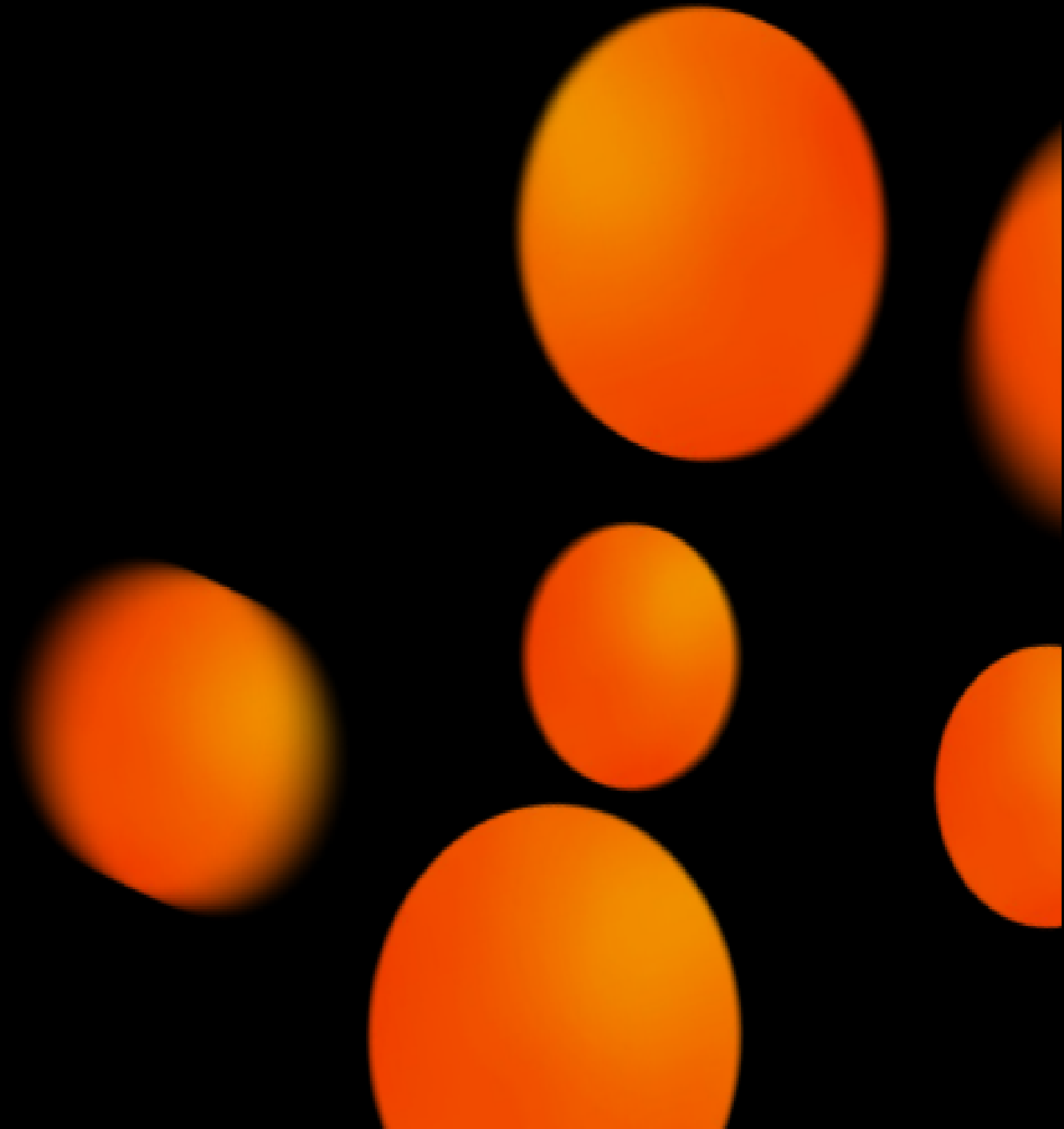
Testes de Segurança

- **O que validam:** Se a API está protegida contra acessos indevidos e ataques. As informações estão seguras?
- **Exemplos:** Tentar acessar dados sem autenticação; usar credenciais inválidas; verificar se diferentes níveis de usuário têm as permissões corretas; testar a exposição de dados sensíveis em mensagens de erro.

Testes de Carga

- **O que validam:** Se a API aguenta um grande volume de requisições e se mantém rápida sob pressão. Qual a sua capacidade máxima?
- **Exemplos:** Simular centenas ou milhares de usuários acessando a API simultaneamente; medir o tempo de resposta em diferentes cenários de carga.

Introdução ao Postman



Postman

- O Postman é uma ferramenta completa e intuitiva para trabalhar com APIs
- Bastante popular para testes de API



POSTMAN

Para que serve?

- **Exploração Rápida:** Envie requisições (pedidos) para APIs e veja as respostas instantaneamente.
- **Testes Manuais:** Perfeito para verificar funcionalidades, erros, e dados de forma ágil.
- **Documentação:** Organize suas requisições em "Coleções", facilitando o reuso e o compartilhamento.
- **Automação (Básica):** Permite escrever pequenos scripts para validar respostas automaticamente e encadear requisições.

Postman

Por que o Postman é tão popular?

- **Interface Amigável:** Fácil de usar, mesmo para quem está começando.
- **Versatilidade:** Suporta diversos tipos de requisições (GET, POST, PUT, DELETE) e autenticações.
- **Colaboração:** Facilita o trabalho em equipe, compartilhando coleções e ambientes.

Criando requisições e validando respostas no Postman

O que são requisições?

HTTP (HyperText Transfer Protocol) é um protocolo usado para comunicação entre clientes (como navegadores ou aplicações) e servidores.

Uma **requisição HTTP** é enviada por um cliente para pedir ou enviar dados ao servidor. Ela é composta por:

→ **Métodos:** definem o tipo da ação

GET: Usado para **solicitar** dados de um servidor. Não deve ter efeitos colaterais.

POST: Usado para **enviar** dados para o servidor e **criar** um novo recurso.

PUT: Usado para **atualizar** um recurso existente por completo.

DELETE: Usado para **remover** um recurso específico do servidor.

→ **URL:** o endereço do recurso

→ **Cabeçalhos (headers):** informações adicionais como autenticação, tipo de conteúdo, etc.

→ **Corpo (body):** usado em métodos como POST/PUT para enviar dados (por exemplo, um JSON)

Prática - O que iremos usar?

→ Postman: <https://www.postman.com/>

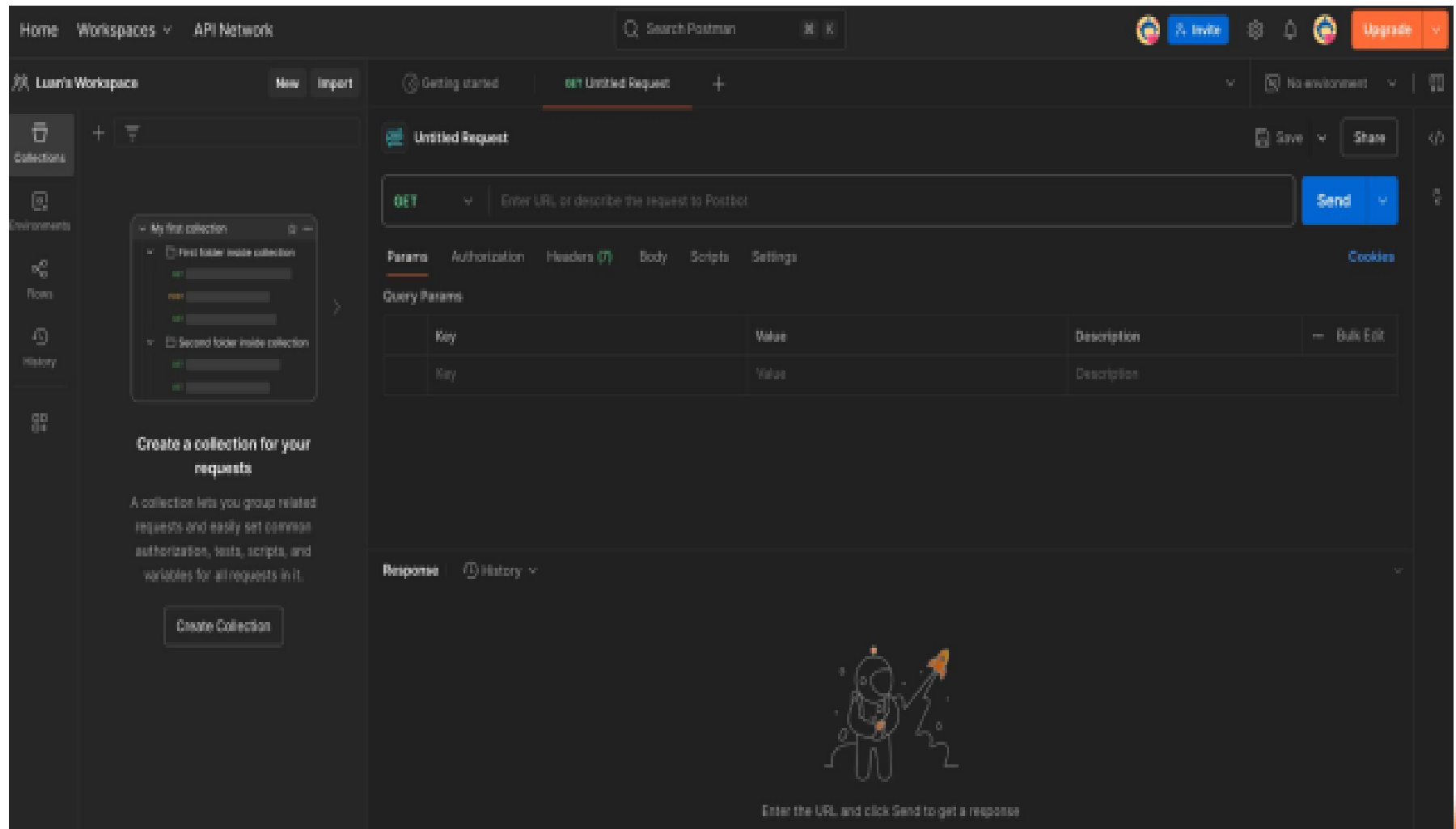
→ API: <https://serverest.dev/>



Configuração inicial

- **Acesse:** postman.com
- **Clique em:** Sign Up for Free → Sign Up with Google → Continue
- **Preencha:** Seu nome
- **Selecione:** Quality Engineer / Tester → Continue
- **Clique em:** Continue with Free Plan → Continue
- **Preencha:** Estudando → Let's go

Interface do Postman



Criando um ambiente

1. No canto superior esquerdo, clique em "+ New" (Novo) e selecione "HTTP Request" (Requisição HTTP).
2. Selecione **"Add new environment"** (Adicionar novo ambiente).
3. Dê um nome ao ambiente, como **"ServeRest Local"**.
4. Na seção **"Variables"** (Variáveis), adicione uma nova variável:
 - 4.1. Variable: `baseUrl`
 - 4.2. Initial value: <https://serverest.dev>
 - 4.3. Current value: <https://serverest.dev>
5. Clique em **"Save"** (Salvar).
6. Selecione o ambiente **"ServeRest Local"** no seletor de ambientes.

Obs: Recomendamos criar uma nova HTTP Request independente para cada exercício, a fim de manter o workspace organizado.

GET – Consultando um usuário

1. Na sua aba de requisição aberta:
 - 1.1. **Método:** Selecione GET
 - 1.2. **URL:** Digite {{baseURL}}/usuarios
2. Clique no botão "Send" (Enviar)

Desafio GET: Tente buscar um usuário específico adicionando o `_id` de um usuário da lista

POST – Cadastrando um usuário

1. Crie uma nova requisição (clique em "+").
 - 1.1. **Método:** Selecione POST
 - 1.2. **URL:** Digite {{baseURL}}/usuarios
 - 1.3. **Headers:** Adicione **Content-Type** com o valor **application/json**
 - 1.4. **Body:** Selecione a opção raw e o tipo JSON. Cole o JSON abaixo, alterando o email para um único e diferente:

```
{  
  "nome": "João da Silva",  
  "email": "joao.silva.SEUNUMEROUNICO@example.com",  
  "password": "senha123",  
  "administrador": "true"  
}
```

2. Clique em **"Send"** (Enviar).

PUT – Atualizando um usuário

1. Crie uma nova requisição (clique em "+").
 - 1.1. **Método:** Selecione PUT.
 - 1.2. **URL:** Digite `{{baseURL}}/usuarios/SEU_ID_DO_USUARIO_CRIADO`
 - 1.3. **Headers:** Adicione **Content-Type** com o valor **application/json**
 - 1.4. **Body:** Selecione a opção raw e o tipo JSON. Cole o JSON abaixo, alterando o email para um único e diferente:

```
{  
  "nome": "Cesar Silva",  
  "email": "cesar.silva.SEUNUMEROUNICO@example.com",  
  "password": "novasenha123",  
  "administrador": "true"  
}
```

2. Clique em **"Send"** (Enviar).

DELETE – Removendo um usuário

1. Crie uma nova requisição (clique em "+").
 - 1.1. **Método:** Selecione DELETE.
 - 1.2. **URL:** Digite {{baseUrl}}/usuarios/**SEU_ID_DO_USUARIO_CRIADO**
2. Clique em "Send" (Enviar).

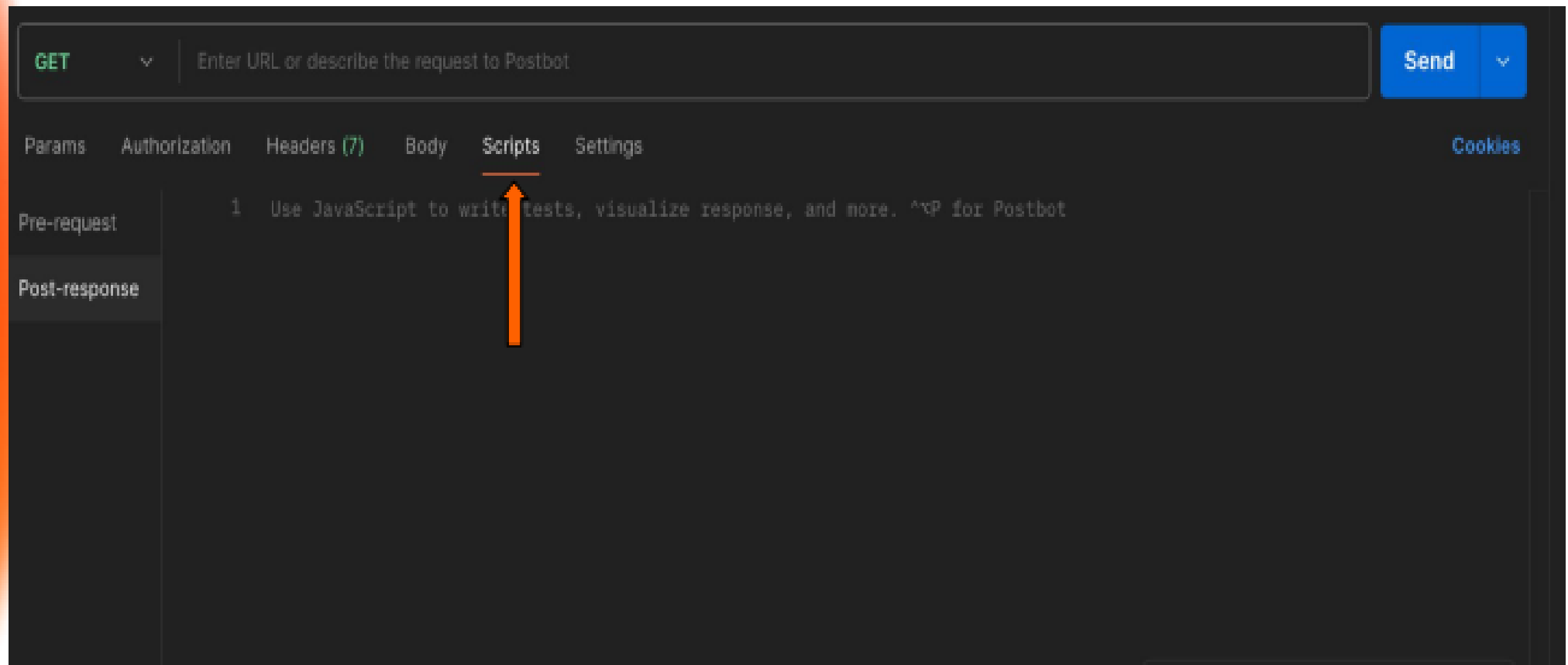
INTERVALO!



15 min.

Scripts automáticos no Postman

Scripts no Postman



Scripts no Postman

→ Usamos o objeto global **pm** (que o Postman oferece) e o **pm.test()** para criar nossas validações.

→ A sintaxe é simples e focada em "o que esperamos":

- `pm.test("Meu teste de status", function () { ... });`
- `pm.response.to.have.status(200);` (Valida o Status Code)
- `pm.expect(pm.response.json().message).to.eql("Sucesso!");` (Valida um texto na resposta JSON)

Scripts no Postman



Validando a requisição GET

→ Vamos validar se a requisição GET retornou sucesso (status 200) e se o corpo da resposta é um array (lista).

Cenário: `GET {{baseUrl}}/usuarios`

```
pm.test("Status da requisição é 200 OK", function () {  
  
    pm.response.to.have.status(200); // Valida se o status HTTP é 200  
  
});  
  
pm.test("Resposta é um array (lista)", function () {  
  
    const responseJson = pm.response.json(); // Pega o corpo da resposta como JSON  
  
    pm.expect(Array.isArray(responseJson.usuarios)).to.be.true;  
  
});
```

Validando a requisição POST

→ Para o POST, queremos garantir que o usuário foi criado com sucesso (status 201) e que a mensagem de sucesso está correta, além de capturar o `_id` do novo usuário.

Cenário: `POST {{baseUrl}}/usuarios`

```
pm.test("Status da requisição é 201 Created", function () {  
  
    pm.response.to.have.status(201); // Valida se o status HTTP é 201  
  
});  
  
pm.test("Mensagem de sucesso ao cadastrar usuário", function () {  
  
    const responseJson = pm.response.json();  
  
    pm.expect(responseJson.message).to.eql("Cadastro realizado com sucesso"); // Valida a  
mensagem específica  
  
});
```

Validando a requisição POST

```
// Captura o ID do usuário criado para usar em outras requisições (PUT/DELETE)  
  
pm.environment.set("usuario_id", pm.response.json()._id); // Armazena o _id em uma variável de ambiente  
  
pm.test("ID do usuário capturado", function() {  
  
    pm.expect(pm.environment.get("usuario_id")).to.exist;  
  
});
```

Validando a requisição PUT

→ Ao atualizar, esperamos um status 200 e uma mensagem de sucesso específica.

Cenário: `PUT {{baseUrl}}/usuarios/{{usuario_id}}`

```
pm.test("Status da requisição é 200 OK", function () {  
  
    pm.response.to.have.status(200); // Valida se o status HTTP é 200  
  
});  
  
pm.test("Mensagem de sucesso ao atualizar usuário", function () {  
  
    const responseJson = pm.response.json();  
  
    pm.expect(responseJson.message).to.eql("Registro alterado com sucesso"); // Valida a mensagem  
  
});
```

Validando a requisição DELETE

→ Para o DELETE, validamos o status 200 e a mensagem de sucesso.

Cenário: DELETE `{{baseUrl}}/usuarios/{{usuario_id}}`

```
pm.test("Status da requisição é 200 OK", function () {  
  
    pm.response.to.have.status(200); // Valida se o status HTTP é 200  
  
});  
  
pm.test("Mensagem de sucesso ao remover usuário", function () {  
  
    const responseJson = pm.response.json();  
  
    pm.expect(responseJson.message).to.eql("Registro excluído com sucesso"); // Valida a mensagem  
  
});
```

**DÚVIDAS?!
:)**





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados